



Introduction to CAG and It's Application

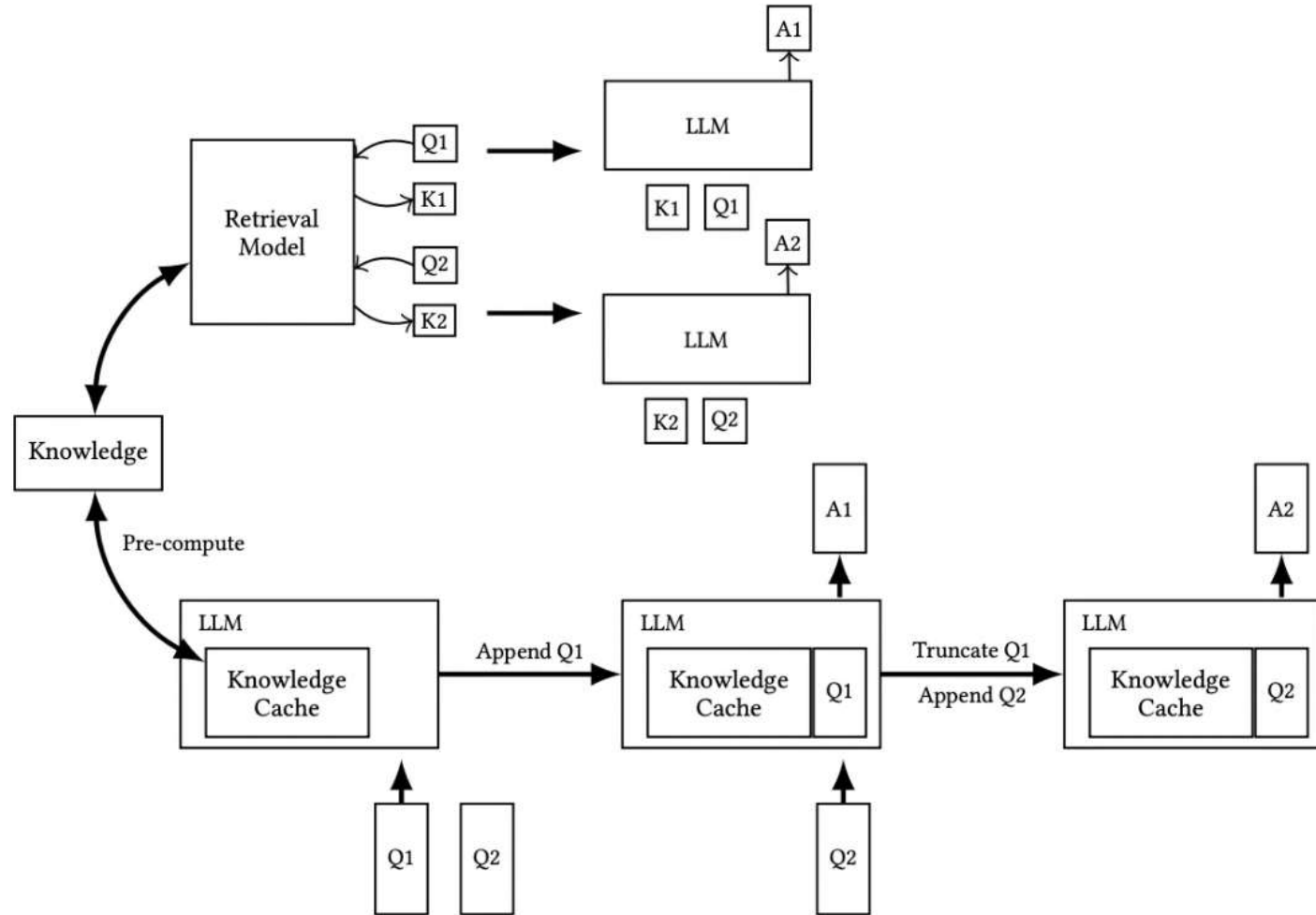
Prof. Osama Abdel Raouf

CAG (**C**ache **A**ugmented **G**eneration)

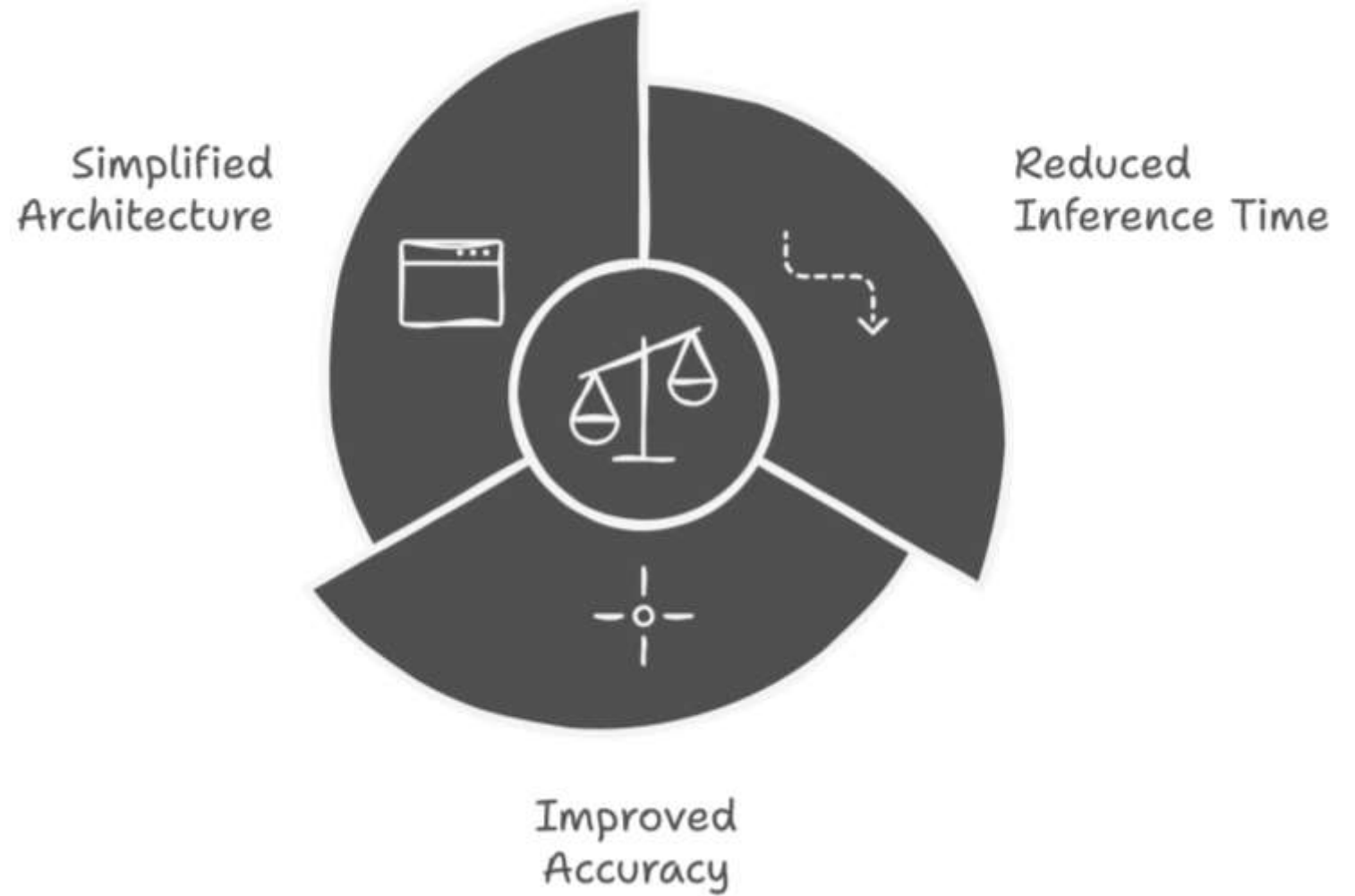
incorporates a cache to store results generated during previous queries or interactions

CAG (Cache Augmented Generation)

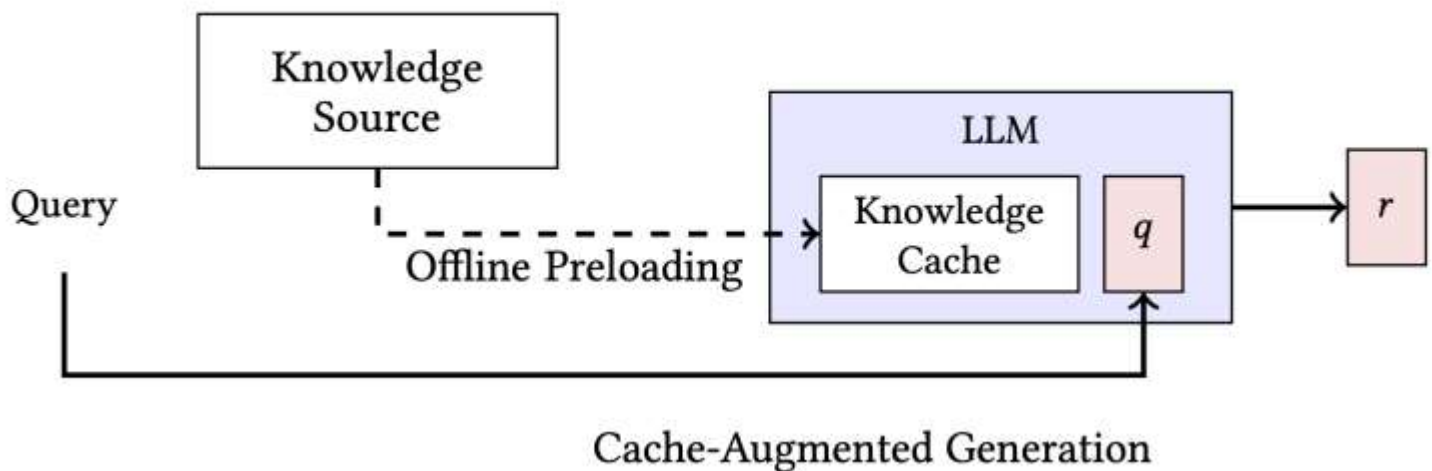
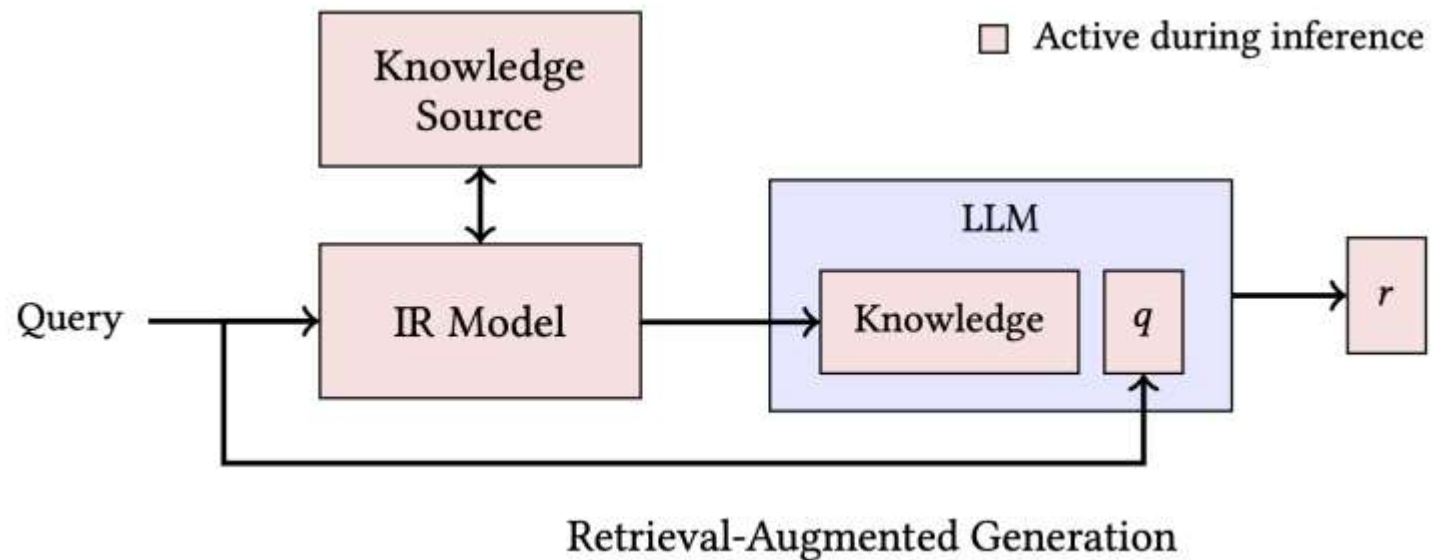
- CAG eliminates the need for real time retrieval by preloading all relevant documents into the model's extended context and precomputing KV caches.



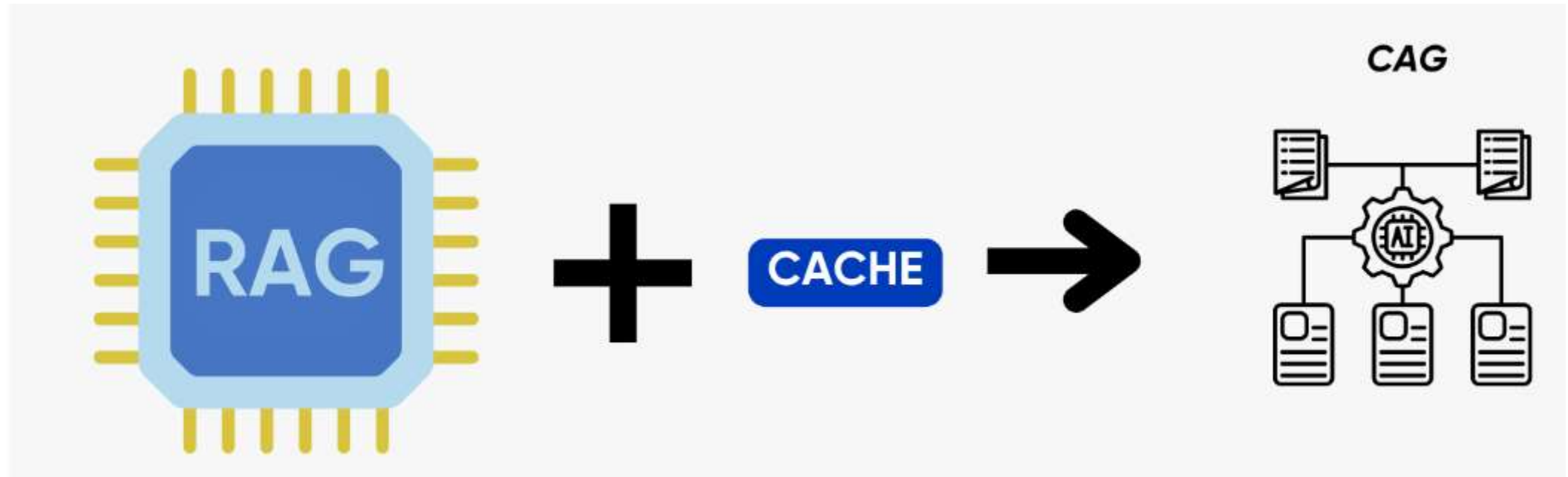
Benefits of CAG Systems



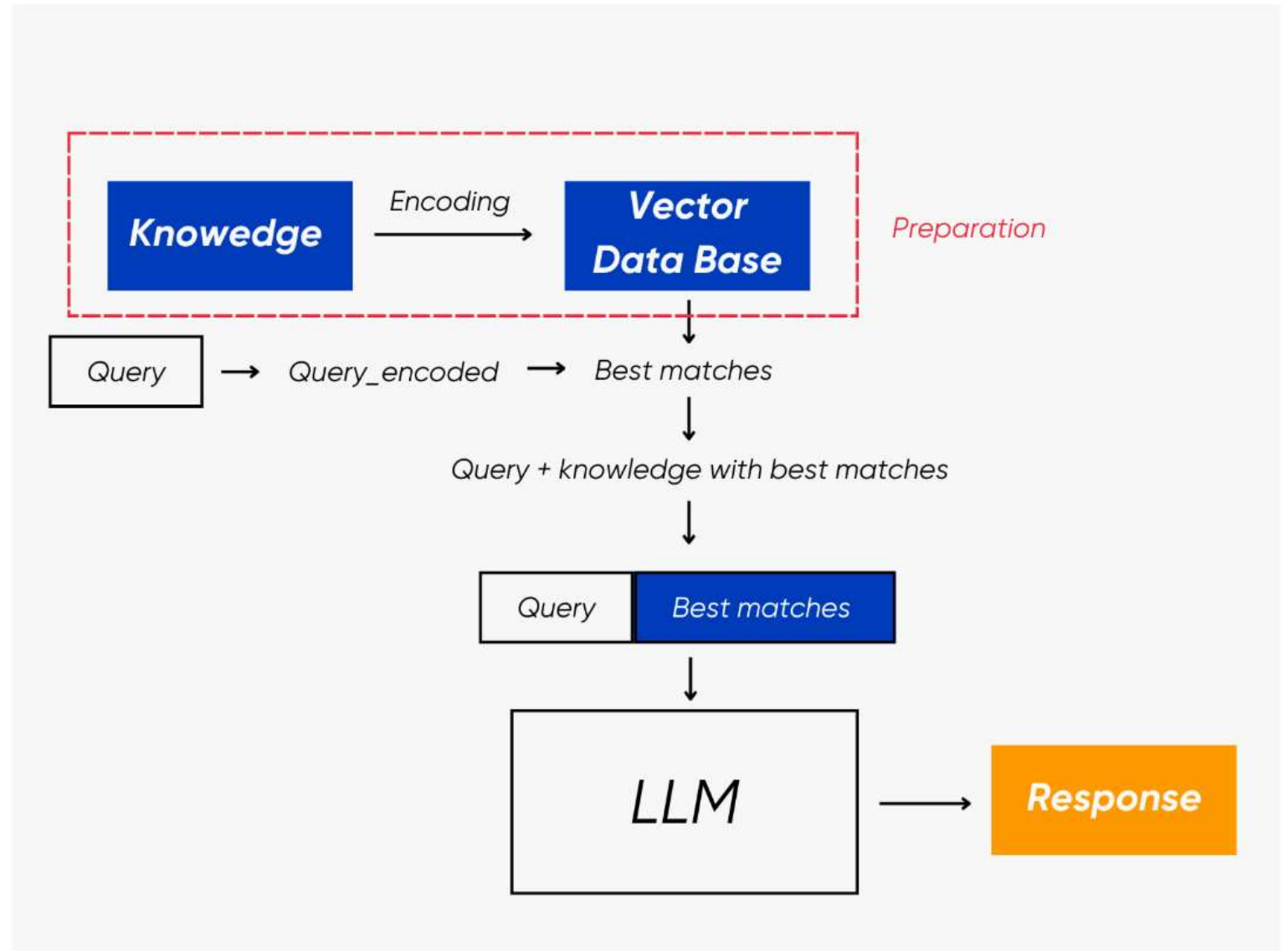
RAG vs CAG



Definition in a simple way



How does CAG work

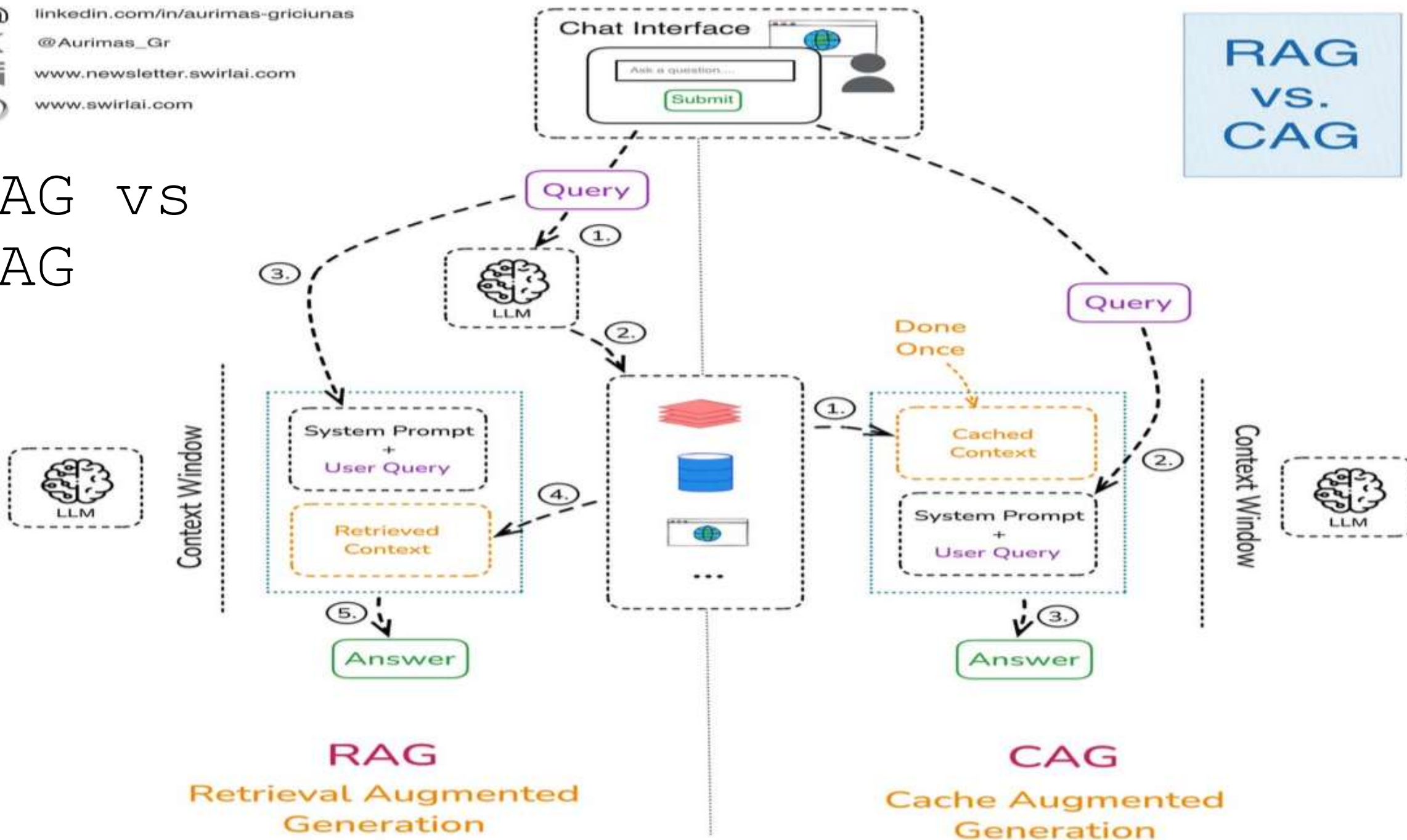




linkedin.com/in/aurimas-griciunas
@Aurimas_Gr
www.newsletter.swirlai.com
www.swirlai.com

RAG vs CAG

RAG vs. CAG



RAG vs CAG

Criteria	RAG (Retrieval-Augmented Generation)	CAG (Contextualized Augmented Generation)
Latency	Higher Latency: RAG requires retrieving external information from a database or search index, which introduces latency.	Lower Latency: CAG typically works with context from the input itself, reducing the need for external retrieval.
Accuracy	Higher Accuracy: By incorporating external, real-time retrieved data, RAG can produce more accurate and informed responses.	Moderate Accuracy: CAG relies on contextual information already present in the input, which may lead to less precise results.
Scalability	Moderate Scalability: RAG can scale well if the external data source (e.g., search engine, knowledge base) is well-managed.	High Scalability: Since CAG works with the context within the model's parameters, it can scale easily without reliance on external sources.
	Higher Data Freshness: RAG	Limited Freshness: CAG

Creating applications using RAG or CAG

1. IT Help Desk Application – CAG (Cache Augmented Generation)

- **Nature of Data:** An IT help desk often deals with **specific, frequently asked questions** or **troubleshooting steps** based on prior incidents, guidelines, and standardized solutions. These queries typically arise from **routine scenarios** and **known patterns** of issues.
- **Real-time Data:** The **contextual information** within the help desk systems (e.g., user's device specs, issue logs, ticket history) is often enough to generate an appropriate response. The answers are **typically static or known solutions** that don't require real-time

Creating applications using RAG or CAG

1. IT Help Desk Application - CAG (Cache Augmented Generation)

Why CAG? Limited context token size

- **Contextual Information:** The model relies on the **immediate context** provided in the help desk query. For instance, an issue described by a user (e.g., "Wi-Fi connectivity problem") can be solved using **historical data**, knowledge bases, or predefined steps without needing to fetch additional data from the web.
- **Efficiency:** With **CAG**, the help desk system can quickly generate responses from stored knowledge, making it ideal for **quick, context-specific troubleshooting**. The speed of response and lower latency are crucial in this scenario to maintain **user satisfaction**.
- **Consistency:** IT issues often have **standard solutions**, and CAG can handle them effectively by leveraging the context (such as system logs, ticket history, and technical context) without needing frequent updates from external data sources.

Creating applications using RAG or CAG

2. Law Research Assistant Application - RAG (Retrieval-Augmented Generation)

- **Nature of Data:** Legal research requires **up-to-date, accurate, and detailed data** such as **case law, statutes, regulations, and precedents**. Legal databases and documents evolve continuously, requiring real-time access to the latest legal texts and rulings.
- **Real-time Data:** Legal research typically requires fetching information from external sources like **legal databases** (e.g., Westlaw, LexisNexis) and **court rulings**, which constantly change. The need for real-time updates makes **CAG** less optimal for

Creating applications using RAG or CAG

2. Law Research Assistant Application - RAG (Retrieval-Augmented Generation)

Why RAG?

- **External Data Retrieval:** RAG is ideal for applications like legal research because it can **retrieve the most relevant documents, case law, and statutes** from external sources, ensuring that the system uses the latest and most accurate information.
- **High Accuracy:** In legal research, the accuracy and **up-to-date relevance** of the information are critical. RAG allows the system to **search real-time databases** to bring in the most relevant and authoritative legal information, ensuring that answers are based on **the most current legal standards**.
- **Complexity Handling:** Legal queries are often complex and involve **multiple sources of information**. With RAG, the system can **retrieve multiple documents** and **synthesize them** to generate accurate, contextually rich responses that align with the nuances of the legal field.
- **Scalability and Freshness:** As legal data continuously evolves (new cases, updated laws, etc.), RAG allows for **real-time retrieval** and scaling across diverse legal topics without retraining the model every time a new legal precedent emerges.

In short note

- **IT Help Desk (CAG)** : Focuses on **known, static solutions** and **contextual data** (e.g., user's problem description, system status). **CAG** is ideal because it can quickly generate answers based on stored information with lower latency.
- **Law Research Assistant (RAG)** : Requires **real-time access** to **constantly evolving legal data**, ensuring **accuracy** and **freshness**. **RAG** excels here because it can retrieve the latest legal documents, making it perfect for **complex, up-to-date**

Hybrid of RAG and CAG

- A **Clinical Decision Support (CDS)** application is a critical tool used in healthcare to help clinicians make informed decisions regarding patient care. The hybrid approach of combining **RAG (Retrieval-Augmented Generation)** and **CAG (Cache Augmented Generation)** could provide a comprehensive solution to enhance the decision-making process. Here's how combining both RAG and CAG would work in a clinical setting:

Hybrid of RAG and CAG

Why a Hybrid of RAG and CAG for Clinical Decision Support CDS?

1. Nature of Data:

- I. **RAG:** Clinical decision-making requires real-time access to a **vast amount of clinical data**, such as **patient records, medical research, clinical guidelines**, and **disease-specific knowledge**. This data evolves frequently as new studies, guidelines, and research findings are published.
- II. **CAG:** Contextual data about the **specific patient's condition**, medical history, **current symptoms**, and **diagnostic results** is crucial for providing accurate recommendations. The system needs to leverage the context of the **patient's specific situation** to generate precise guidance.

2. Key Benefits of Hybrid Approach:

- I. **Real-time Updates (RAG):** Using RAG allows the CDS system to **retrieve up-to-date** research findings, clinical guidelines, drug interactions, and case studies from external databases (e.g., PubMed, medical journals, clinical guidelines). These resources are constantly updated with new evidence and protocols.
- II. **Contextual Relevance (CAG):** CAG ensures that recommendations are tailored to the **individual patient** by using **contextual data** such as

Hybrid of RAG and CAG

Aspect	RAG	CAG	Hybrid Approach
Data Retrieval	Retrieves external medical data : clinical guidelines, research papers, drug interactions, and evidence-based practices from up-to-date medical databases.	Uses patient-specific data (e.g., symptoms, medical history, current diagnosis) to provide recommendations.	Combines real-time medical data retrieval with personalized patient context for decision support.
Accuracy	Provides accurate, evidence-based recommendations by pulling from the latest research and clinical databases.	Provides contextualized recommendations based on the specific needs of the patient, making it more relevant .	High accuracy : Pulls accurate, up-to-date data and filters it through personalized patient context .
Scalability	Highly scalable due to the ability to retrieve vast amounts of data from external sources (medical research, databases).	Scalable in terms of managing large volumes of patient-specific data and applying it in a personalized way.	Provides scalability in both general knowledge retrieval and personalized context application .
Data Freshness	Real-time freshness : Can fetch current research, new drug approvals , and updated guidelines from external sources.	Context-specific freshness : Patient data may change over time (e.g., new test results or evolving conditions).	Combines fresh, real-time medical information with context-sensitive patient updates for accurate decisions.
Latency	Higher latency due to data retrieval from external sources (searching medical databases).	Lower latency as it works with internal data already available in the system (patient records).	Hybrid approach: Balanced latency by using quick contextual data while occasionally fetching external data when needed.

How the Hybrid Approach Enhances CDS Applications

1. Real-Time Data with Personalization:

- I. **RAG** ensures that the system always incorporates **current medical guidelines, new research, and updated treatments**, which are vital in clinical settings where recommendations can quickly become outdated.
- II. **CAG** ensures that the **clinical decisions are based on the specific context of the patient** (e.g., their medical history, symptoms, current test results), making the system **adaptive and personalized**.

2. Filling the Gaps:

- I. In certain cases, **clinical guidelines or databases** may not have immediate answers to specific or rare conditions. The **contextualization** from **CAG** helps fill this gap by providing **patient-centric recommendations**.
- II. When new medical findings emerge, **RAG** ensures the system can **incorporate these findings** into its decision-making process, keeping it relevant.

How the Hybrid Approach Enhances CDS Applications

3. Multidimensional Decision Making:

I. **RAG** retrieves **generalizable medical knowledge** such as **best practices for diseases, drug interactions, or emerging treatments.**

II. **CAG** applies that knowledge to a **specific patient's context**, such as choosing the appropriate dosage of medication based on a patient's weight, age, or organ function.

III. Combining both approaches allows the system to **integrate broad medical knowledge** with **personalized care**, thus making the clinical decisions more accurate and relevant.

Optional Enhancements

1

Query Expansion:

Rewrite queries for better retrieval.

2

Hybrid Search:

Combine BM25 (keyword) + vector search.

3

Re-ranking:

Reorder retrieved chunks (e.g., Cohere Rerank) .

Installation

```
pip install -r ./requirements.txt
```

Preparation

Important

download the required `squad` and `hotpotqa` datasets by curl script

```
sh ./downloads.sh
```

Important

create `.env` file by `.env.template` and input the keys required

```
cp ../.env.template ../.env
```


Usage

`rag.py` is
for RAG
Experiment
t

`kvcache.py`
y is for
CAG
Experiment
t

Parameter Usage -- kvcache.py

- `--kvcache` : "file"
- `--dataset` : "hotpotqa-train" or "squad-train"
- `--similarity` "bertscore"
- `--modelname` : "meta-llama/Llama-3.1-8B-Instruct"
- `--maxKnowledge` : "", int, select how many document in dataset, explanation in Note
- `--maxParagraph` : 100
- `--maxQuestion` int, max question number, explanation in Note
- `--randomSeed` : "", int, a random seed number
- `--output` : "", str, output filepath string
- `--usePrompt` , add this parameter if not using CAG knowledge cache acceleration

Example -- kvcache.py

```
python ./kvcache.py --kvcache file --dataset "squad-train" --similarity bertscore \  
    --maxKnowledge 5 --maxParagraph 100 --maxQuestion 1000 \  
    --modelname "meta-llama/Llama-3.1-8B-Instruct" --randomSeed 0 \  
    --output "./result_kvcache.txt"
```

```
python ./kvcache.py --kvcache file --dataset "squad-train"  
--similarity bertscore \  
    --maxKnowledge 5 --maxParagraph 100 --maxQuestion 1000  
\  
    --modelname "meta-llama/Llama-3.1-8B-Instruct" --  
randomSeed 0 \  
    --output "./result_kvcache.txt"
```

Note:

--maxKnowledge parameter notice:

Note

Approximate Tokens count corresponding to knowledge document size of "squad-train" and "hotpotqa-train" dataset.

datasets=("squad-train")

- when $k = 3$, tokens = 21,000
- when $k = 4$, tokens = 32,000
- when $k = 7$, tokens = 50,000

datasets=("hotpotqa-train")

- all $k = 7405$ article, tokens = 10,038,084
- when $k = 1$, tokens = 1,400
- when $k = 16$, tokens = 22,400
- when $k = 24$, tokens = 33,667
- when $k = 32$, tokens = 44,800
- when $k = 48$, tokens = 64,000
- when $k = 64$, tokens = 85,000
- when $k = 80$, tokens = 106,000

--maxQuestion parameter notice:

- when using "squad-train" dataset, 1 knowledge has average 150 questions
- when using "hotpotqa-train" dataset, 1 knowledge has 1 question

Tip

Since 1 document in "hotpoqa-train" dataset has only 1 question, it may not satisfy large-scale evaluation. Multiple evaluation could be a relatively better approach.

Docker

To build the docker image, run

```
docker build -t my-cag-app .
```



and to run the container, run this for GPU users

```
docker run --gpus all -it --rm my-cag-app
```



for CPU users.

if the .env file details were empty while building you will get error similar to this below

```
Traceback (most recent call last):  
File "/app/./kvcache.py", line 35, in <module>  
    env = validate_env_variables()  
          ^^^^^^^^^^^^^^^^^^^^^^^  
  
File "/app/./kvcache.py", line 31, in validate_env_variables  
    raise ValueError(f"Missing required environment variable: {key}")  
ValueError: Missing required environment variable: HF_TOKEN
```

so ensure you populate the `.env` file before building the docker image

Note that the `he CMD` directive in the Dockerfile runs the `kvcache.py` script by default. You can override this in the `docker run` command if you'd like to execute other scripts like `rag.py`. For example:

```
docker run --gpus all -it --rm my-cag-app python ./rag.py --index "bm25" --dataset "hotpotqa-train" --s
```

A 3D paper speech bubble is centered on a solid blue background. The bubble is white with a subtle drop shadow, giving it a three-dimensional appearance. Inside the bubble, the words "THANK YOU!" are printed in a bold, blue, sans-serif font. The entire scene is framed by a white border, which is itself set against a larger blue background with a decorative blue L-shaped corner element in the top-left corner.

THANK YOU!